ALLIANCE UNIVERSITY
Alliance School of Advanced Computing

Micro–Project Report Format

Template for Micro–project **Report** after Execution in ***not more thant 15 pages***
(to be submitted at end of semester along with seminar presentation for ***assessing*** the attainment of the stated COs and Competency)

Course Code: E1CSA25479 **Course Title**: Data Structures

Name of Course In-charge: Dr Chetan Shelke

Student Team Members

1.Namitha
2.Ritika
3.Chandana
4.Hari Satya
5.Priyanka

ARCHAEOLOGICAL SITE EXPLORATION SYSTEM

1.0 Rationale

Exploring an archaeological site with several interconnected locations can be difficult when the locations and their connecting paths are not represented in an organized way. The Archaeological Site Exploration System addresses this problem by modelling selected locations of Hampi as vertices of a graph and the paths between them as edges. This representation provides a simple and visual way to demonstrate how graph data structures can be applied to a real-world exploration problem.

The project is implemented in C++ and provides functions to display locations, display their connections, search for a particular location, and traverse the site using Breadth-First Search (BFS) and Depth-First Search (DFS). BFS uses a queue to explore connected locations level by level, while DFS uses recursion to explore deeper connections before backtracking. A simple web-based demonstration is also associated with the project to make the site representation easier to view. Thus, the micro-project connects classroom concepts of graphs, arrays, queues, recursion, searching and traversal algorithms with an archaeological-site scenario.

2.0 Competency of the Course

The basic purpose of this one sentence is to help the student to develop the following *macro-level professional competency*, which is an entry-level *industry-specific main task* that is to be attained by the student through this course and its attainment assessed through the *micro-project*.

Evaluate the performance of data structure in various applications

3.0 Course Outcomes Addressed in this Micro Project

Evaluate the suitability of searching and sorting algorithms for efficient data processing

Evaluate the suitability of the tree data structures for efficient data management.

Evaluate the suitability of graph data structures for efficient data traversal.

4.0 Information Collection

Information for the micro-project was collected from the Data Structures learning material, classroom notes, C++ practice resources, and reliable online resources related to graphs, searching, BFS and DFS. The project proposal identifies Data Structures Through C++ by Yashavant Kanetkar as a principal textbook for understanding graph representation and searching concepts. The team used these concepts to decide how archaeological locations could be represented as nodes and how the paths between them could be represented as graph edges.

The team first studied the basic terminology of graphs, including vertices, edges, adjacency and graph traversal. The searching requirement was then considered as a comparison of a user-entered location with the locations stored in an array. For traversal, BFS was studied as a queue-based method in which a starting location is marked visited, inserted into a queue, removed from the front and followed by its unvisited neighbours. DFS was studied as a recursive method that visits an unvisited neighbour, continues deeper, and backtracks when there are no more unvisited neighbours.

The selected Hampi model contains six locations: Entrance, Hemakuta Hill, Krishna Temple, Sasivekalu Ganesha, Achyutaraya Temple and Vitthala Temple. The connections are stored using an array of vectors, which provides an adjacency-list style graph representation. The collected information was then converted into algorithms, a flowchart and C++ functions. The algorithms and flowchart documents supplied for the project describe the individual steps for displaying locations and connections, searching, BFS and DFS

5.0 Theory and Design of the Micro Project

5.1 Graph Representation

A graph is used as the main data structure because the selected archaeological locations are naturally interconnected. Each location is represented by a node (vertex), while a connection between two locations is represented by an edge. In the C++ program, the locations are stored in a string array named place and the graph connections are stored in an array of vectors named graph.

The project uses six nodes numbered from 0 to 5. The node numbers are associated with the following locations

Node	Location	Connections
0	Entrance	Hemakuta Hill, Krishna Temple
1	Hemakuta Hill	Entrance, Sasivekalu Ganesha
2	Krishna Temple	Entrance, Achyutaraya Temple
3	Sasivekalu Ganesha	Hemakuta Hill, Vitthala Temple
4	Achyutaraya Temple	Krishna Temple, Vitthala Temple
5	Vitthala Temple	Sasivekalu Ganesha, Achyutaraya Temple

5.2 Data Structures Used

Graph: Represents the locations and their connections.

Array: Stores the names of the six archaeological locations.

Vector: Stores the adjacency list for each graph node.

Queue: Used by BFS to process locations in level order.

Recursion: Used by DFS to continue exploring connected locations and backtrack.

Boolean visited array: Prevents the same location from being visited repeatedly during BFS and DFS.

5.3 Algorithms and Design Logic

Display Locations: Traverse the location array from the first to the last element and display each location with its node number. Display Connections: Traverse each graph node and display the locations contained in its adjacency list. Search Location: Read a location name, compare it with each stored location, and report whether a match is found.

BFS: Start at the selected node, mark it visited, insert it into a queue, then repeatedly remove the front node and add its unvisited neighbours.
DFS: Start at the selected node, mark and display it, recursively visit an unvisited neighbour, and backtrack when no unvisited neighbour remains

5.4 Program Structure

The program contains separate functions for each major operation. displayPlaces() prints all stored locations, displayConnections() prints the adjacency relationships, searchPlace() performs a sequential search, BFS() performs queue-based traversal, and DFS() performs recursive traversal. The main() function initializes the graph and repeatedly displays a menu until the user selects Exit

6.0 Methodology Followed

1. Project selection and problem definition: The team selected archaeological-site exploration as a practical scenario for demonstrating graph-based Data Structures concepts.
2. Site modelling: Six Hampi locations were selected and assigned node numbers from 0 to 5.
3. Graph design: Connections between the locations were planned and stored as an adjacency-list style graph.
4. Algorithm development: Separate logic was prepared for displaying locations, displaying connections, searching, BFS and DFS.
5. Flowchart preparation: The main menu flow and individual operation flows were represented in a flowchart.
6. C++ implementation: The graph and algorithms were implemented in C++ using arrays, vectors, queue and recursion.
7. Testing: The program was run with different menu choices and starting nodes to check the traversal and search outputs.
8. Correction and documentation: Coding readability, comments, error handling for invalid node numbers, and project documentation were improved.
9. Web demonstration: A simple HTML/CSS/JavaScript representation was associated with the project to provide an interactive demonstration of the archaeological-site model

7.0 Outputs of the Micro–project

7.1 C++ Program Output and Features

The completed C++ program provides the following menu-driven operations:

Display Locations – prints all six Hampi locations with node numbers.

Display Connections – prints the graph connections for every location.

Search Location – searches for an exact location name and reports whether it is found.

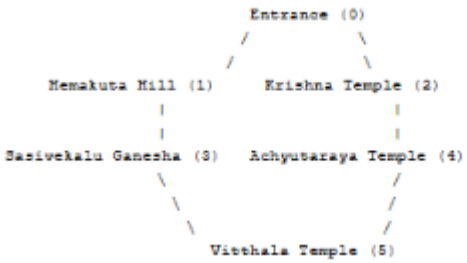
BFS Traversal – accepts a starting node and explores the reachable graph using a queue.

DFS Traversal – accepts a starting node and explores the graph recursively.

Exit – terminates the program with a thank-you message.

7.2 Sample Graph / Site Design

Graph model of the selected Hampi locations



The graph is designed as a connected cycle-like structure. From Entrance, the traversal can proceed towards Hemakuta Hill or Krishna Temple. The two branches continue through Sasivekalu Ganesha and Achyutaraya Temple respectively and meet at Vitthala Temple. This structure provides a clear example for demonstrating both breadth-first and depth-first exploration

7.3 Sample Traversal Results

For example, if node 0 (Entrance) is selected as the starting node, the program follows the adjacency lists in the order stored in the graph

Operation	**Starting Node**	**Expected traversal/search behaviour**
BFS	0 – Entrance	Entrance → Hemakuta Hill → Krishna Temple → Sasivekalu Ganesha → Achyutaraya Temple → Vitthala Temple
DFS	0 – Entrance	Entrance → Hemakuta Hill → Sasivekalu Ganesha → Vitthala Temple → Achyutaraya Temple → Krishna Temple
Search	Krishna Temple	Location Found: Krishna Temple
Search	Invalid name	Location Not Found.

7.4 Findings

A graph is suitable for representing locations connected by paths because it directly models vertices and edges.
BFS provides level-wise exploration and requires a queue.
DFS explores one branch deeply before backtracking and can be implemented naturally using recursion.
An array provides a simple way to store and compare the names of locations for searching.
The adjacency-list representation keeps the connection information organized and easy to traverse.
Separating the program into functions makes the implementation easier to understand, test and maintain.

7.5 Conclusion

The Archaeological Site Exploration System successfully demonstrates the application of Data Structures and algorithms to a simulated archaeological-site scenario. The project models Hampi locations as graph nodes and their connections as edges, then uses searching, BFS and DFS to explore the model. The final program provides a simple menu-driven interface and demonstrates how theoretical Data Structures concepts can be translated into a practical C++ application.

7.6 Limitations

The site is a simplified model and does not represent the complete real-world geography of Hampi.
The program uses a fixed set of six locations.
The search operation is an exact string comparison, so different spellings or partial names are not handled.
The C++ program is primarily a demonstration of Data Structures and does not calculate real walking distance or travel time.
The graph connections are predefined rather than being entered dynamically by the user

8.0 Benefits of this Micro–project

The project gives practical experience in applying graph data structures to an understandable real-world scenario. It strengthens understanding of vertices, edges, adjacency-list representation, searching and graph traversal. Implementing BFS with a queue and DFS with recursion also helps connect algorithmic theory with C++ programming. The menu-driven program improves problem-solving, modular programming and testing skills, while the simple web demonstration makes the concept easier to visualize.

9.0 Area of Future Improvement

The system can be extended by adding more Hampi locations and allowing the user to create or modify graph connections dynamically. A future version could include weighted edges to represent actual distances or estimated travel time and could implement shortest-path algorithms such as Dijkstra's algorithm. The search feature could be improved to support partial or case-insensitive names. The web interface could also provide animated BFS/DFS traversal, interactive node selection, maps, images and additional information about each archaeological location.

10.0 References

[1] Kanetkar, Yashavant – Data Structures Through C++, for graph, searching and C++ Data Structures concepts.

[2] Alliance University, Alliance School of Advanced Computing – Micro–Project Proposal: Archaeological Site Exploration System.

[3] Alliance University, Alliance School of Advanced Computing – Algorithms: Archaeological Site Exploration.

[4] Alliance University, Alliance School of Advanced Computing – Flowchart: Archaeological Site Exploration.

[5] C++ classroom notes and practice resources used by the project team.

[6] Project GitHub Pages website: https://5509-srr.github.io/Archaeological-Site-Exploration-DS

C++ code:

```
[1]  #include <iostream>
[2]  #include <vector>
[3]  #include <queue>
[4]  using namespace std;
[5]
[6]  string place[6] = {
[7]      "Entrance",

[8]      "Hemakuta Hill",
[9]      "Krishna Temple",
[10]     "Sasivekalu Ganesha",
[11]     "Achyutaraya Temple",
[12]     "Vitthala Temple"
[13] };
[14]
[15] vector<int> graph[6];
[16]
[17] void displayPlaces() {
[18]     cout << "\n--- Hampi Locations ---\n";
[19]     for (int i = 0; i < 6; i++) {
[20]         cout << i << ". " << place[i] << endl;
[21]     }
[22] }
[23]
[24] void displayConnections() {
[25]     cout << "\n--- Site Connections ---\n";
[26]     for (int i = 0; i < 6; i++) {
```

```
[27]        cout << place[i] << " -> ";
[28]        for (int j : graph[i])
[29]            cout << place[j] << "  ";
[30]        cout << endl;
[31]    }
[32] }
[33]
[34] void searchPlace() {
[35]    string name;
[36]    cout << "\nEnter location: ";
[37]    cin.ignore();
[38]    getline(cin, name);
[39]
[40]    for (int i = 0; i < 6; i++) {
[41]        if (place[i] == name) {
[42]            cout << "Location Found: " << place[i] << endl;
[43]            return;
[44]        }
[45]    }
[46]
[47]    cout << "Location Not Found.\n";
[48] }
[49]
[50] void BFS(int start) {
[51]    bool visited[6] = {};
[52]    queue<int> q;
[53]
[54]    visited[start] = true;
[55]    q.push(start);
[56]
[57]    cout << "\nBFS: ";
[58]
[59]    while (!q.empty()) {
[60]        int current = q.front();
[61]        q.pop();
[62]
[63]        cout << place[current] << " -> ";
[64]
[65]        for (int next : graph[current]) {
[66]            if (!visited[next]) {
[67]                visited[next] = true;
[68]                q.push(next);
[69]            }
[70]        }
[71]    }
[72]
[73]    cout << "END\n";
[74] }
[75]
[76] void DFS(int current, bool visited[]) {
[77]    visited[current] = true;
[78]    cout << place[current] << " -> ";
[79]
[80]    for (int next : graph[current]) {
[81]        if (!visited[next])
[82]            DFS(next, visited);
[83]    }
[84] }
[85]
[86] int main() {
[87]
[88]    graph[0] = {1, 2};
[89]    graph[1] = {0, 3};
```

```
    graph[2] = {0, 4};
    graph[3] = {1, 5};
    graph[4] = {2, 5};
    graph[5] = {3, 4};

    int choice, start;

    do {
        cout << "\n===== ARCHAEOLOGICAL SITE EXPLORATION =====\n";
        cout << "1. Display Locations\n";
        cout << "2. Display Connections\n";
        cout << "3. Search Location\n";
        cout << "4. BFS Traversal\n";
        cout << "5. DFS Traversal\n";
        cout << "6. Exit\n";
        cout << "Enter your choice: ";
        cin >> choice;

        if (choice == 1) {
            displayPlaces();
        }
        else if (choice == 2) {
            displayConnections();
        }
        else if (choice == 3) {
            searchPlace();
        }
        else if (choice == 4) {
            cout << "Enter starting node (0-5): ";
            cin >> start;

            if (start >= 0 && start < 6)
                BFS(start);
            else
                cout << "Invalid node.\n";
        }
        else if (choice == 5) {
            cout << "Enter starting node (0-5): ";
            cin >> start;

            if (start >= 0 && start < 6) {
                bool visited[6] = {};
                cout << "\nDFS: ";
                DFS(start, visited);
                cout << "END\n";
            }
            else {
                cout << "Invalid node.\n";
            }
        }
        else if (choice == 6) {
            cout << "\nThank you!\n";
        }
        else {
            cout << "\nInvalid choice.\n";
        }

    } while (choice != 6);

    return 0;
}
```

Github Repository Link:
Your site is live at https://5509-srr.github.io/Archaeological-Site-Exploration-DS/

11.0 Applications and Practical Significance

The Archaeological Site Exploration System demonstrates how Data Structures can be applied to practical problems involving interconnected locations. The graph representation provides an organized way to store locations and their connections.

11.1 Archaeological Site Exploration

The system represents selected locations of Hampi as vertices and the paths connecting them as edges. This model helps users understand the relationships between different locations and explore them using graph traversal algorithms.

11.2 Navigation Systems

Graphs are widely useful in navigation systems because roads, junctions, and destinations can be represented as vertices and edges. Search and traversal techniques help process the relationships between these places. With additional distance information, the system could be extended to support route-planning algorithms.

11.3 Social Networks

In social networks, people can be represented as vertices and their relationships as edges. Graph traversal can help explore connections between people and identify reachable users within a network.

11.4 Computer Networks

Computer networks contain interconnected computers and devices. Graphs can represent these devices and their communication links. Traversal algorithms can help examine network connectivity and explore connected devices.

11.5 Educational Applications

The project is useful for learning Data Structures because it connects theoretical concepts with a practical example. Students can understand vertices, edges, adjacency lists, queues, recursion, searching, BFS, and DFS through one application.

11.6 Scope for Practical Development

The current implementation is a simplified demonstration and does not calculate actual walking distances or travel times. In a future version, the system could include additional archaeological locations, location descriptions, photographs, weighted connections, and an interactive map. A shortest-path algorithm such as Dijkstra's algorithm could be considered if distance information were added.

11.7 Overall Significance

This micro-project demonstrates that Data Structures are useful beyond classroom exercises. By modelling an archaeological site as a graph, the project provides a foundation for understanding how algorithms organize and process connected information. The same fundamental ideas can be adapted to navigation, network analysis, and other applications involving interconnected entities.